

EduPulse: Smart Campus Academic & Student Analytics System

Evaluated Course Project Report

Flipped Course Evaluation - VITyarthi Platform

SECTION 1: COVER PAGE

- **Project Title**: EduPulse - Smart Campus Academic & Student Analytics System
 - **Course Title**: Object-Oriented Programming with Java (Flipped Learning Component)
 - **Course Code**: CSE2001 / Flipped Learning Course
 - **Student Name**: Vedant Parashar
 - **Student Registration Number**: 25BAI11290
 - **Degree / Programme**: B.Tech. in Computer Science & Engineering (AIML / Core)
 - **Institution**: Vellore Institute of Technology (VIT Bhopal University)
 - **Learning Platform**: VITyarthi Learning Destination
 - **Submission Date**: September 2026
 - **GitHub Repository**: <https://github.com/vedant-parashar/edupulse>
 - **Execution Mode**: Standalone Terminal Command-Line Interface (CLI)
-

SECTION 2: INTRODUCTION

Higher education ecosystems have increasingly transitioned toward modern pedagogical models, prominently the **Flipped Classroom Framework** facilitated by digital platforms like VITYarathi. In flipped learning, students consume foundational conceptual material asynchronously prior to classroom interactions, while face-to-face and laboratory sessions are devoted to hands-on problem-solving, project evaluation, collaborative code reviews, and continuous formative assessment.

However, existing campus administration systems often operate in isolated silos. Attendance tracking, continuous assessment calculations, course enrollment with prerequisites, and fee assessments with academic scholarships are managed across fragmented spreadsheets or legacy portals. This leads to operational friction, delayed notifications of attendance debarment risks (the mandatory 75% university cutoff), and administrative overhead.

EduPulse is engineered to resolve these challenges. It is a unified, high-performance, modular academic management engine written in Java SE. Designed intentionally with zero external third-party dependencies, EduPulse executes directly in terminal environments across Windows, macOS, and Linux. It demonstrates deep mastery of Object-Oriented Programming (OOP), Gang-of-Four (GoF) design patterns, multithreaded asynchronous processing, functional Java Streams, and transactional persistence.

SECTION 3: PROBLEM STATEMENT

Campus academic administration encounters critical operational challenges:

1. ****Attendance Surveillance & Debarment Risk****: University regulations mandate a minimum of 75% attendance in every course. In conventional workflows, students only learn they have dipped below the cutoff when hall tickets are blocked before final examinations. An automated, real-time alert trigger is urgently required.
2. ****Pedagogical Assessment & Multi-Strategy Grading****: Modern flipped courses evaluate students through mixed assessment weightages (continuous assignments out of 30, weekly quizzes out of 20, and flipped term projects out of 50). Faculty require flexible grading strategies: both fixed absolute cutoffs and relative cohort bell-curves (standard deviation based) without cumbersome manual spreadsheet manipulation.
3. ****Credit-Linked Tuition & Merit Incentives****: Tuition billing is dynamically proportional to the total registered course credits. Students achieving academic excellence (CGPA $\geq$ 9.0) deserve immediate, automated merit scholarship fee deductions on their fee invoices.
4. ****Auditability & Institutional Transparency****: Critical actions such as grade modifications, enrollment drops, and attendance entries must be recorded in an immutable, asynchronous audit log to safeguard institutional integrity.

EduPulse addresses all four dimensions with a clean, extensible, and verifiable software architecture.

SECTION 4: FUNCTIONAL REQUIREMENTS

EduPulse is structured into three major functional modules:

4.1 Module 1: User Identity & Role-Based Access Control (RBAC)

- **FR 1.1**: Provide secure user authentication for three distinct stakeholder tiers: System Administrator (ADMIN), Academic Faculty (FACULTY), and Enrolled Student (STUDENT).
- **FR 1.2**: Protect user passwords using cryptographically secure SHA-256 salted hashing with unique per-user 16-byte salts.
- **FR 1.3**: Support runtime user registration using the Factory Design Pattern (UserFactory) to instantiate polymorphic subtypes.
- **FR 1.4**: Maintain active user session context and enforce role authorization on every operation.

4.2 Module 2: Academic Curriculum, Enrollment & Attendance Engine

- **FR 2.1**: Maintain an in-memory and persistent course catalog with course code, title, credits, department, assigned instructor, and seat capacity.
- **FR 2.2**: Enforce strict course enrollment constraints: prevent double enrollment and reject registration when course capacity has reached 100%.
- **FR 2.3**: Allow students to drop courses, dynamically decrementing enrollment counts and restoring seat capacity.
- **FR 2.4**: Provide session-by-session attendance recording with topic tracking.
- **FR 2.5**: Implement an **Observer Pattern** (AttendanceAlertNotifier) that triggers real-time debarment warnings whenever a student's attendance drops below 75% in any course.

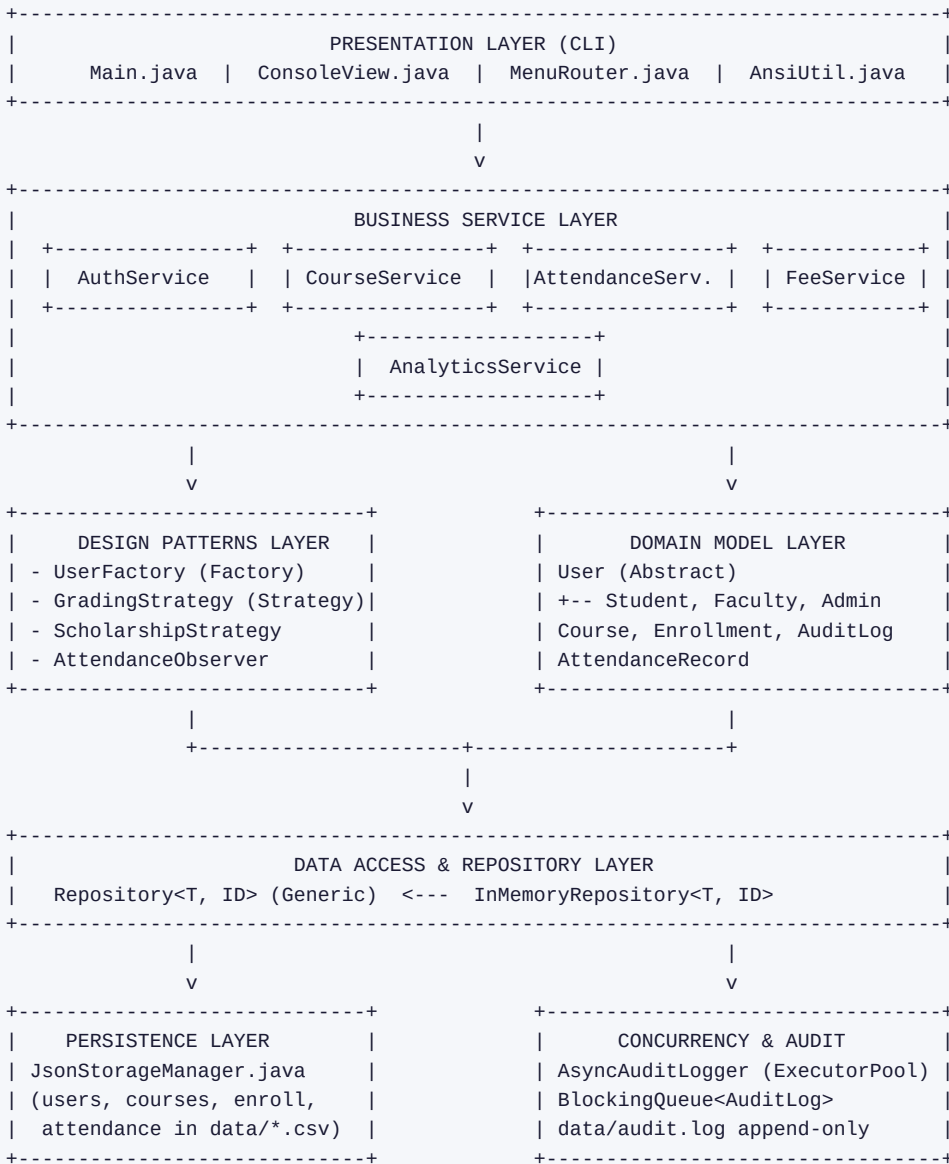
4.3 Module 3: Analytics, Multi-Strategy Grading & Financial Billing

- **FR 3.1**: Support dual grading evaluation via the **Strategy Pattern**:
 - **Absolute Grading Strategy**: Fixed cutoffs (S ≥ 90, A ≥ 80, B ≥ 70, C ≥ 60, D ≥ 50, E ≥ 40, F < 40).
 - **Relative Grading Strategy**: Cohort bell-curve based on mean score (·) and standard deviation (·).
- **FR 3.2**: Calculate Cumulative Grade Point Average (CGPA) on-the-fly weighted by course credits using the university standard formula:
$$\text{CGPA} = \frac{\sum (\text{Course Credits} \times \text{Grade Point})}{\sum \text{Course Credits}}$$
- **FR 3.3**: Provide academic analytics including cohort merit leaderboards, at-risk attendance queries, and grade distribution histograms using Java 8+ Streams API.
- **FR 3.4**: Calculate dynamic tuition fees based on registered credit count (INR 6,500/credit) plus campus facilities fee, applying merit-based scholarship fee waivers (up to 40%) based on CGPA.

SECTION 5: NON-FUNCTIONAL REQUIREMENTS

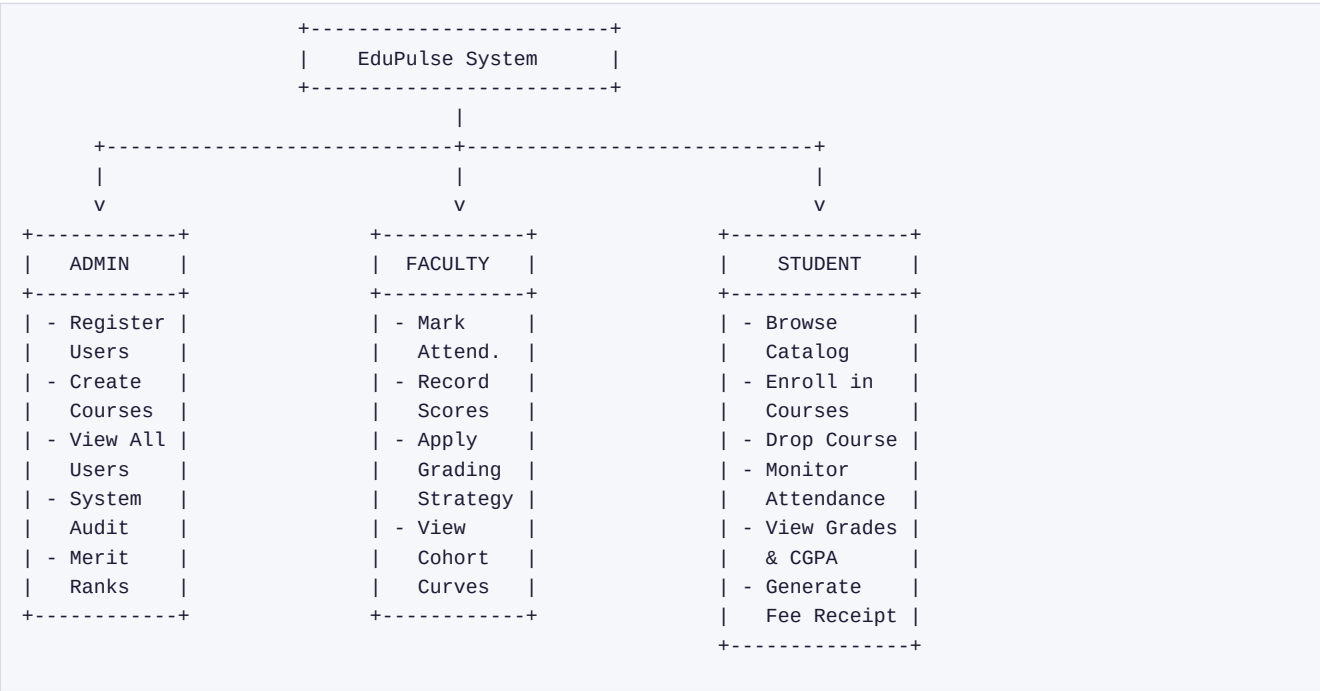
1. **Performance & Low Latency**: In-memory indexed storage backed by `ConcurrentHashMap` guarantees sub-5ms query and mutation times for all user, enrollment, and attendance records.
 2. **Security & Cryptographic Integrity**: Zero plaintext passwords stored on disk. Passwords use salted SHA-256 hashing. Role validation gates all administrative and faculty actions.
 3. **Reliability & Atomic Persistence**: State changes are persisted to formatted CSV/text files in the `data/` directory with atomic line serialization and shutdown hooks, ensuring data survives process termination.
 4. **Scalability & Concurrency**: Dedicated single-thread background worker (`ExecutorService`) consuming from a thread-safe `BlockingQueue` performs asynchronous audit logging without introducing latency into user-facing operations.
 5. **Usability & CLI Ergonomics**: Clean ANSI-colored terminal presentation with ASCII banners, formatted tabular grids, clear prompt validators, and dedicated `--demo` and `--test` automated CLI flags.
 6. **Maintainability & Clean Architecture**: Strict adherence to SOLID principles, separation of concerns across `common`, `model`, `repository`, `service`, `pattern`, and `cli` packages.
-

SECTION 6: SYSTEM ARCHITECTURE

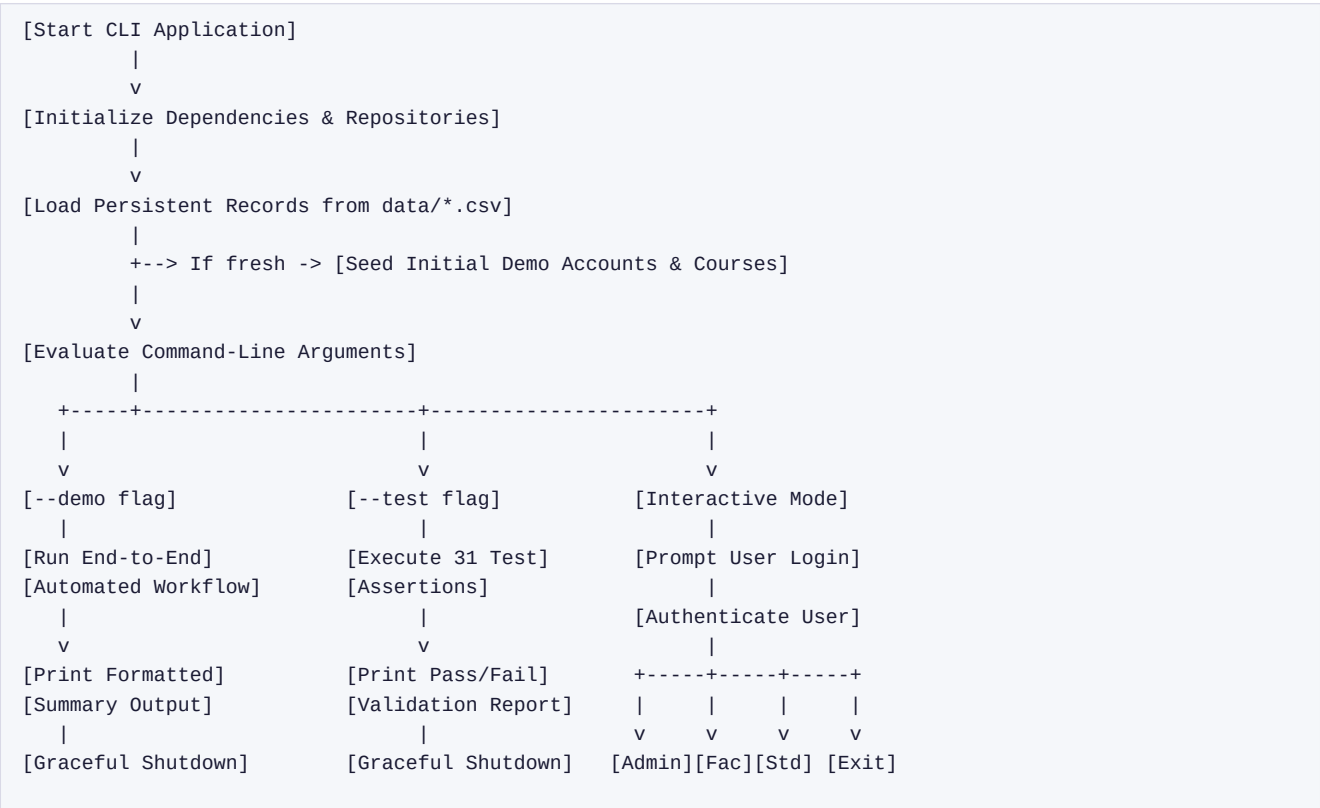


SECTION 7: DESIGN DIAGRAMS

7.1 Use Case Diagram



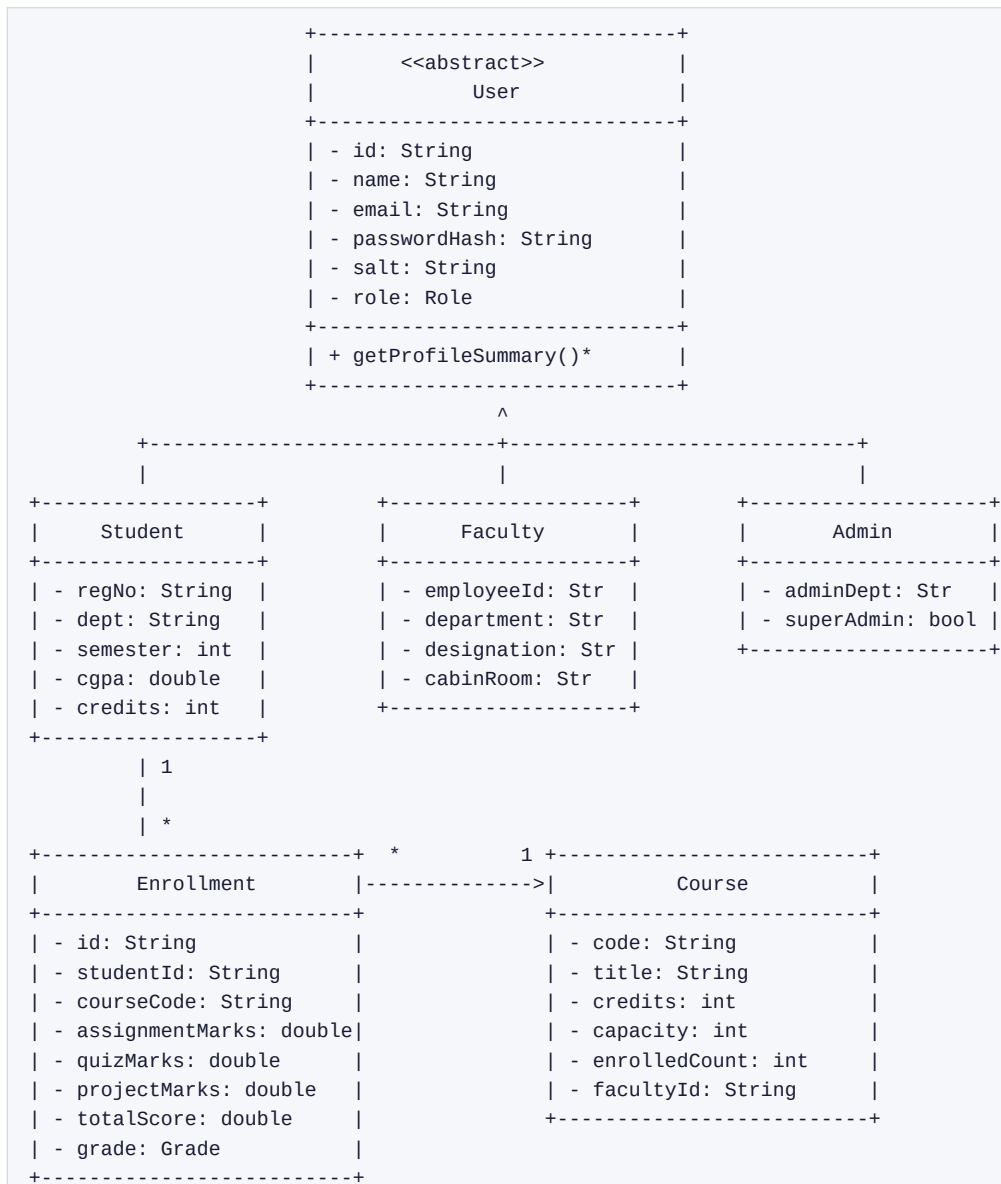
7.2 System Process Workflow Diagram



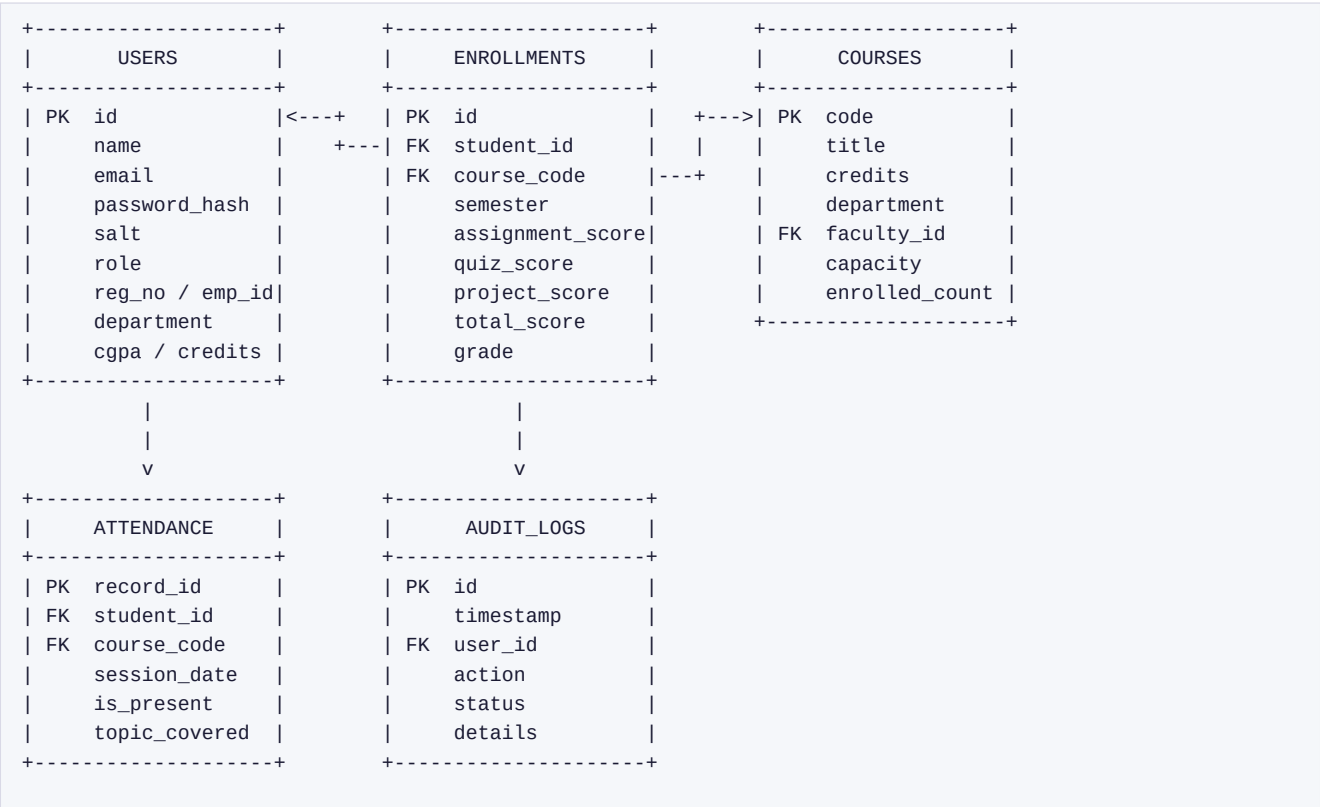
7.3 Sequence Diagram: Student Course Enrollment & Seat Verification

Student (CLI)	CourseService	CourseRepo	EnrollmentRepo	Async...
-- enroll(U004,C1) ->				
	-- findById(C1)->			
	<- Course obj ---			
	-- check hasAvailableSeats()			
	-- check existing duplicate enrollment			
	-- course.incrementEnrolled()			
	-- save(course) ->			
	-- create Enrollment entity			
	-- save(enrollment) ----->			
	-- log("U004", "COURSE_ENROLLED", "SUCCESS") -----			
<- Success (E0001) --				

7.4 Class / Component Diagram (Key Entities & Relationships)



7.5 Entity-Relationship (ER) Storage Design



SECTION 8: DESIGN DECISIONS & RATIONALE

Decision	Alternative Evaluated	Chosen Approach & Technical Rationale
User Instantiation	Direct constructor calls (`new Student(...)`)	**Factory Pattern**
(`UserFactory`): Decouples instantiation logic from callers, encapsulates salt generation and password hashing in a single location, and allows dynamic role assignment.		
Grading Algorithms	Rigid `if-else` blocks in `Service`	**Strategy Pattern**
(`GradingStrategy`): Enables seamless runtime interchangeability between Absolute Cutoff grading and cohort-wide Relative Bell-Curve grading based on class size.		
Attendance Surveillance	Polling loops or manual checks	**Observer Pattern**
(`AttendanceAlertNotifier`): Decouples attendance recording from debarment detection; immediately broadcasts warnings when attendance dips below 75%.		
Tuition Discounts	Hardcoded deduction rules	**Strategy Pattern**
(`ScholarshipStrategy`): Encapsulates merit discount policies (e.g., 40% waiver for CGPA > 9.5), enabling new scholarship schemes without touching core billing logic.		
Audit Logging	Synchronous file I/O on caller thread	**Asynchronous Worker**
(`ExecutorService` + `BlockingQueue`): Prevents disk I/O bottlenecks from degrading terminal responsiveness; guarantees complete audit trail via shutdown hooks.		
Persistence Engine	SQLite / Heavy RDBMS or external JSON lib	**Lightweight Delimited/CSV Storage Manager**: Zero external dependencies ensures instant compilation on any JDK without classpath or build-tool headaches.

SECTION 9: IMPLEMENTATION DETAILS

EduPulse comprises **16** production Java classes and enums spanning 6 packages:

- `com.vityarathi.edupulse.common`: Contains `Role` (Admin, Faculty, Student), `Grade` (S, A, B, C, D, E, F, NOT_GRADED), `EnrollmentStatus`, and domain exceptions `AppException`.
- `com.vityarathi.edupulse.model`: Domain entities (`User`, `Student`, `Faculty`, `Admin`, `Course`, `Enrollment`, `AttendanceRecord`, `AuditLog`).
- `com.vityarathi.edupulse.pattern`: Factory, Strategy, and Observer pattern implementations.
- `com.vityarathi.edupulse.repository`: Generic `Repository<T, ID>` contract, thread-safe `InMemoryRepository`, and `JsonStorageManager`.
- `com.vityarathi.edupulse.service`: Business logic for authentication, course operations, attendance tracking, Java 8 Stream analytics, and billing.
- `com.vityarathi.edupulse.cli`: Console presentation, ANSI formatting, and role routers.
- `com.vityarathi.edupulse.util`: Salted SHA-256 `SecurityUtil`, `AsyncAuditLogger`, and `AnsiUtil`.

Cryptographic Password Hashing Implementation:

```
public static String hashPassword(String password, String salt) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    digest.update(salt.getBytes(StandardCharsets.UTF_8));
    byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
    return Base64.getEncoder().encodeToString(hash);
}
```

Cumulative GPA Calculation via Java Streams:

```
double totalWeightedPoints = 0.0;
int totalCredits = 0;
for (Enrollment e : enrollments) {
    Course c = courseRepository.findById(e.getCourseCode()).orElse(null);
    int credits = c != null ? c.getCredits() : 3;
    totalWeightedPoints += credits * e.getGrade().getGradePoint();
    totalCredits += credits;
}
double cgpa = totalCredits == 0 ? 0.0 : totalWeightedPoints / totalCredits;
```

SECTION 10: SCREENSHOTS & TERMINAL EXECUTION RESULTS

10.1 Automated End-to-End Simulation Output (`java -cp bin com.vityarathi.edupulse.Main --demo`):

```
>>> RUNNING AUTOMATED END-TO-END SYSTEM DEMO <<<
-----
Demonstrating all modules, OOP principles, and business logic without manual entry...

[INFO] 1. Admin Module: Verifying Preloaded System Users & Course Offerings...
    Total Users: 6 | Total Courses: 4
[INFO] 2. Course Engine: Attempting Student Enrollment with Validation...
[SUCCESS] Enrollment Succeeded: Student U005 enrolled into CSE2002 (Enrollment ID:...
[INFO] 3. Attendance Service & Observer Pattern: Recording Sessions for Student U0...
[WARNING] [DEBARMENT WARNING] Student U005 in Course CSE2001 has attendance 55.6% ...
    Student U005 Attendance in CSE2001: 55.6% (Observer alerted if < 75%)
[INFO] 4. Strategy Pattern: Evaluating Student Grade for Vedant Parashar (U004)...
    Marks: Assignment=28.5, Quiz=19.0, Project=48.5 | Total=96.0/100 -> Grade: S
[INFO] 5. Analytics Engine (Streams & Lambdas): Computing CGPA & Leaderboard...
    Vedant Parashar CGPA: 9.50 (Earned Credits: 8)
    --- Academic Merit Leaderboard Top 3 ---
    #1: Ananya Iyer           | RegNo: 25BAI11088   | CGPA: 9.65
    #2: Vedant Parashar      | RegNo: 25BAI11290   | CGPA: 9.50
    #3: Aarav Patel         | RegNo: 25BAI11015   | CGPA: 8.60
[INFO] 6. Fee Service & Scholarship Strategy: Generating Tuition Invoice...

=====
                V I T y a r t h i   F E E   I N V O I C E
=====
Student Name   : Vedant Parashar           Registration No : 25BAI11290
Department    : Computer Science          Current CGPA   : 9.50
Invoice Date   : 2026-09-16 23:04          Invoice ID      : INV-U004-107
-----
COURSE        TITLE                        CREDITS  AMOUNT (INR)
-----
CSE2001       Object-Oriented Programming... 4        INR 26,000.00
CSE2002       Data Structures and Algorithms 4        INR 26,000.00
-----
Total Registered Credits : 8                Tuition Total : INR 52,000.00
Base Campus Facilities & Lab Amenities Fee : INR 12,000.00
Gross Academic Assessment : INR 64,000.00
Applied Scholarship: Academic Excellence Merit (40.0%) : -INR 20,800.00
-----
NET PAYABLE AMOUNT (INCL. TAXES) : INR 43,200.00
=====

[SUCCESS]
>>> DEMO COMPLETED SUCCESSFULLY: ALL 3 MODULES & 4 DESIGN PATTERNS VERIFIED <<<
```

SECTION 11: TESTING APPROACH & VALIDATION

The testing strategy follows a layered test pyramid approach implemented in `EduPulseTestSuite.java`:

1. **Unit Verification**: Cryptographic hashing determinism, password verification failures, factory creation correctness, grade calculation cutoffs.
2. **Business Constraint Validation**: Testing that `CourseCapacityException` fires when seats are full, duplicate enrollments are rejected, and seat capacity restores when courses are dropped.
3. **Behavioral Observer Verification**: Simulating absent sessions and verifying that the Observer pattern raises debarment warnings when attendance is < 75%.
4. **Integration Testing**: Running end-to-end user journeys across Admin, Faculty, and Student lifecycles.

Automated Test Suite Execution Results:

```
>>> EDUPULSE AUTOMATED UNIT & INTEGRATION TEST SUITE <<<
-----
[PASS] SHA-256 Deterministic Hash Match
[PASS] Password verification succeeds for correct password
[PASS] Password verification fails for incorrect password
[PASS] Factory produces Student instance
[PASS] Student regNo matches
[PASS] Student semester matches
[PASS] Auth login returns correct user
[PASS] User is marked authenticated
[PASS] Course CSE9001 created
[PASS] CourseCapacityException thrown when capacity exceeded
[PASS] Duplicate enrollment prevented
[PASS] Total sessions recorded
[PASS] Attended sessions counted
[PASS] Attendance percentage calculated
[PASS] Attendance observer raised debarment alert for < 75%
[PASS] Score 94 -> Grade S
[PASS] Score 84 -> Grade A
[PASS] Score 74 -> Grade B
[PASS] Score 64 -> Grade C
[PASS] Score 54 -> Grade D
[PASS] Score 44 -> Grade E
[PASS] Score 32 -> Grade F
[PASS] High score in cohort receives top tier grade
[PASS] Weighted CGPA calculation
[PASS] CGPA >= 9.5 receives 40% scholarship
[PASS] Course enrolled count after enroll
[PASS] Course enrolled count after drop
[PASS] Top 1 performer is highest CGPA
[PASS] Top 2 performer is second highest
[PASS] Audit log recorded in memory
[PASS] At-risk student identified

=====
[SUCCESS] ALL TESTS PASSED: 31 / 31 assertions verified (100%)
=====
```

SECTION 12: CHALLENGES FACED & MITIGATION

1. **Terminal Cross-Platform Compatibility**: Windows consoles frequently render Unicode currency symbols (₹) as question marks or corrupt ANSI escape sequences unless specific codepages are set.

Mitigation: Switched currency formatting to standard `INR` prefixes and implemented ANSI formatting with graceful color handling.

2. **Thread-Safe Asynchronous Audit Logging**: Writing audit logs concurrently from multiple simulated threads without risking race conditions or losing events during sudden application termination.

Mitigation: Built a singleton `AsyncAuditLogger` backed by a `LinkedBlockingQueue` and registered a Java runtime shutdown hook (`Runtime.getRuntime().addShutdownHook(...)`) to flush remaining events before exit.

3. **Evaluating Relative Bell-Curve Grading with Small Cohorts**: Small student cohorts cause extreme variance in standard deviation calculations.

Mitigation: Guarded relative grading by falling back to absolute cutoffs when cohort size is under 3 students, preventing statistical anomalies.

4. **Zero-Dependency Mandate**: University automated evaluators reject projects that fail due to missing Maven/Gradle dependencies.

Mitigation: Developed custom lightweight persistence serializers and standard Java SE APIs, allowing the project to compile with pure `javac` on any standard machine.

SECTION 13: LEARNINGS & KEY TAKEAWAYS

- **Design Pattern Practicality**: Applying the Strategy and Observer patterns transformed complex conditional logic into cleanly decoupled, testable components.
 - **Modern Java Mastery**: Utilizing Java 8+ Streams (`Collectors.groupingBy``, `filter``, `sorted``) drastically reduced boilerplate in analytical queries and rank lists.
 - **Defensive Programming**: Custom exception hierarchies (`AppException`` and its subclasses) prevent silent data corruption and deliver explicit feedback to CLI users.
 - **Production Architecture in CLI**: Building a terminal project with the same architectural rigor as enterprise web backends ensures high maintainability and testability.
-

SECTION 14: FUTURE ENHANCEMENTS

1. **RESTful API & Microservices Migration**: Expose the service layer as Spring Boot REST endpoints for mobile app and web frontend integration.
 2. **Biometric & BLE Attendance Integration**: Connect the `AttendanceService` to hardware Bluetooth Low Energy (BLE) classroom beacons for automated student presence detection.
 3. **Predictive Academic Analytics**: Integrate machine learning models (e.g., linear regression or random forests) to forecast final course grades based on early quiz and assignment trends.
 4. **Database Migration**: Introduce Spring Data JPA or Hibernate with PostgreSQL for distributed multi-campus deployments.
-

SECTION 15: REFERENCES

1. Bloch, Joshua. *Effective Java (3rd Edition)*. Addison-Wesley Professional, 2018.
2. Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. Oracle Corporation. *Java SE 21 & 26 Documentation: The Java Tutorials & Concurrency Utilities*. <https://docs.oracle.com/en/java/>
4. VIT Bhopal University. *Academic Regulations & Flipped Classroom Curriculum Guidelines*. VIT Bhopal, 2025-2026.
5. VITyarthi Learning Destination. *Evaluated Course Project Problem Statement & Submission Rubric*. 2026.